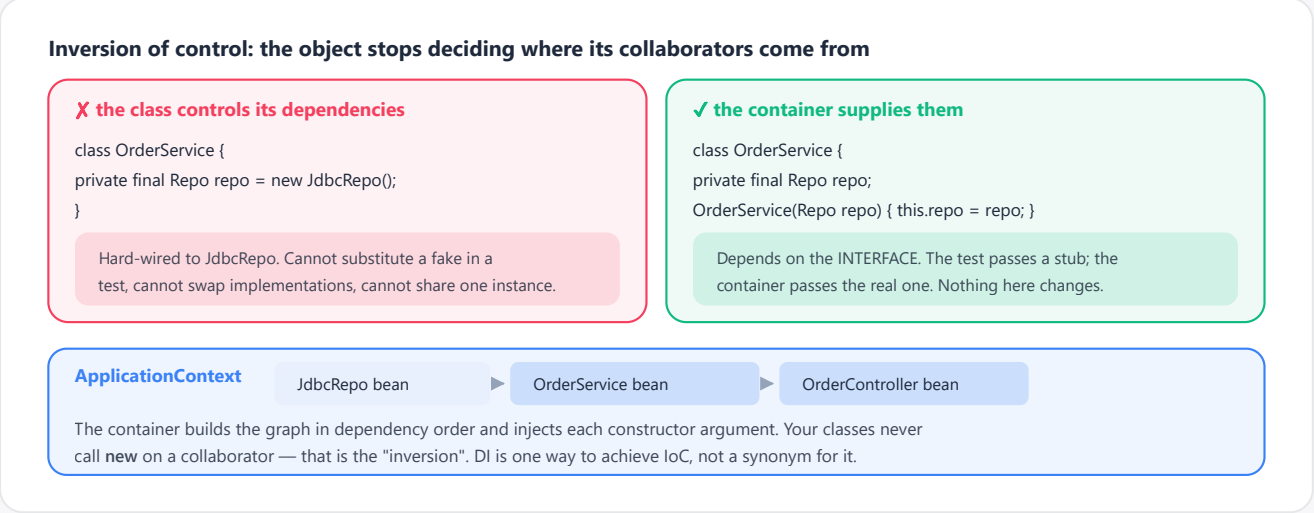
IoC and Dependency Injection

Why it matters

Every Spring question eventually reduces to this one. If you can explain what inversion of control actually inverts — and why constructor injection is not merely a style preference — the rest of the framework follows. It is also the question where a memorised definition is most obvious: "IoC means Spring creates your objects" is true and explains nothing.

The concept, simply

Inversion of control means the framework, not your code, decides when objects are created and wired together. **Dependency injection** is the specific technique Spring uses to achieve it: a class declares what it needs, and something else supplies it.



The inversion is that the class stops calling new. It declares a need; the container satisfies it. That single change is what makes the class testable, swappable and shareable.

The three injection styles, in the order you should prefer them:

Constructor	**Use this.** Dependencies can be `final`, the object is never half-built, and it w
Setter	Only for genuinely optional dependencies
Field (`@Autowired` on a field)	Avoid — cannot be `final`, hides dependencies, needs reflection to test

How it works

Spring scans for stereotype annotations, builds a graph of bean definitions, then instantiates them in dependency order and injects each constructor argument. **Since Spring 4.3, a class with a single constructor needs no `@Autowired` at all** — which is why modern Spring code has almost no injection annotations in it.

The stereotypes are all `@Component` underneath; the distinctions are for readability and for one real behaviour:

- `@Component` — generic bean.
- `@Service` — business logic. Semantic only.
- `@Repository` — data access. **Not** semantic only: it enables exception translation, converting vendor-specific `SQLException`s into Spring's `DataAccessException` hierarchy.
- `@Controller` / `@RestController` — web layer.

Code examples

JAVA

Copy

```java
// ✓ Constructor injection — no annotation needed with one constructor.
@Service
public class OrderService {
    private final OrderRepository repository;      // final: guaranteed se
    private final PaymentGateway gateway;
```

```
        this.gateway = gateway;
    }
}

// The test needs no Spring at all:
var service = new OrderService(new InMemoryOrderRepository(), new FakeGate

// ✗ Field injection — the constructor tells you nothing, the fields cannc
//   be final, and constructing it in a plain unit test is impossible with
//   reflection or a Spring context.
@Service
public class OrderService {
    @Autowired private OrderRepository repository;
}
```

Resolving ambiguity, which is the most common wiring failure:

JAVA Copy

```
// Two implementations of one interface → NoUniqueBeanDefinitionException.
public interface PaymentGateway { }
@Component class StripeGateway implements PaymentGateway { }
@Component class PaypalGateway implements PaymentGateway { }

// ✓ Option 1 — mark a default.
@Component @Primary class StripeGateway implements PaymentGateway { }

// ✓ Option 2 — name the one you want at the injection point.
public OrderService(@Qualifier("paypalGateway") PaymentGateway gateway) {

// ✓ Option 3 — inject them all and choose at run time. Often the best
//   design, because it removes the branching from configuration.
public OrderService(List<PaymentGateway> gateways) { … }
public OrderService(Map<String, PaymentGateway> byBeanName) { … }
```

Circular dependencies, and why the fix is design rather than configuration:

```
// A needs B, B needs A. Constructor injection makes this a STARTUP FAILUR
// which is correct — the cycle is a design problem.
// Since Spring Boot 2.6 it is rejected by default rather than silently al

// ✗ The usual "fixes" hide the problem:
//   spring.main.allow-circular-references=true
//   @Lazy on one of the constructor parameters
//   switching to field injection so the cycle is resolved after construct

// ✓ Real fixes: extract the shared behaviour into a third bean that both
//   depend on, or publish an event instead of calling back. If A and B ne
//   each other, they are usually one concept split in the wrong place.
```

Under the hood

Why constructor injection matters beyond style comes down to three concrete properties. Fields can be `final`, so the object is immutable and safely publishable across threads (Chapter 8's memory-model guarantee). The object can never exist in a half-initialised state, because it is not constructed until every dependency is available. And a constructor with eight parameters is a visible, uncomfortable signal that the class does too much — field injection hides exactly that smell behind a tidy-looking class.

Spring beans are singletons by default, and that is a threading contract. One instance serves every request thread concurrently, so any mutable field on a `@Service` is shared mutable state. This is the single most common Spring concurrency bug: an instance field used to pass data between methods works perfectly under manual testing and corrupts data under load. Keep beans stateless; pass state as parameters.

Spring's singleton is not the singleton pattern. It means "one instance per `ApplicationContext`", not one per JVM — two contexts have two instances, and the class has no private constructor or static accessor.

Common mistakes

- **Field injection.** Hides dependencies, prevents `final`, and makes plain unit tests impossible.
- **Mutable state on a singleton bean.** Shared across every request thread.
- **Working around circular dependencies.** The cycle is the bug; `@Lazy` just delays it.
- **Injecting the container.** `ApplicationContextAware` or `getBean()` is service-locator, not injection.
- **Adding `@Autowired` to a single constructor.** Unnecessary since Spring 4.3.
- **Assuming `@Service` and `@Repository` are interchangeable.** `@Repository` enables exception translation.
- **Injecting a prototype into a singleton.** It is resolved once, so you get the same instance forever.

Interview lens

Q: "What is inversion of control?"

A: Normally an object decides which concrete collaborators it uses by calling `new`. IoC inverts that: the object declares what it needs and the container supplies it, so control over creation and wiring moves out of the class. Dependency injection is Spring's mechanism for doing it — the two are related but not synonyms, since IoC is the principle and DI one implementation.

Q: "Which injection type do you use and why?"

A: Constructor, essentially always. It lets dependencies be `final`, so the bean is immutable and safe to share across threads; the object cannot exist half-built; and it is constructible in a plain unit test with no Spring at all. It also makes an over-large dependency list visible, which field injection conveniently hides. Since Spring 4.3 a single constructor needs no `@Autowired`.

Q: "Are Spring beans thread-safe?"

A: The container is, but your beans are not automatically. The default scope is singleton — one instance per context, serving every request thread concurrently —

also that Spring's "singleton" means one per `ApplicationContext`, not the Gang of Four singleton pattern.

Q: "Two beans implement the same interface. What happens?"
A: `NoUniqueBeanDefinitionException` at startup. I resolve it with `@Primary` for a sensible default, `@Qualifier` at the injection point when I need a specific one, or — often best — by injecting `List<T>` or `Map<String, T>` and selecting at run time, which turns a configuration decision into ordinary code.

Q: "How do you handle a circular dependency?"
A: By removing it. With constructor injection Spring fails at startup, and since Boot 2.6 that is the default — which is the right behaviour, because a cycle means two beans are mutually responsible for each other. The real fixes are extracting the shared logic into a third bean, or replacing a direct call with an event. Using `@Lazy` or enabling circular references hides a design problem.

Q: "Difference between `@Component`, `@Service` and `@Repository`?"
A: All are `@Component` underneath and are all picked up by scanning. `@Service` is purely semantic. `@Repository` is the one with real behaviour — it enables persistence exception translation, wrapping vendor-specific `SQLException`s in Spring's `DataAccessException` hierarchy so callers are not coupled to the database.

Cheat-card

- IoC = the container controls creation and wiring. **DI is how Spring does it.**
- **Constructor injection**: `final` fields · never half-built · testable without Spring · exposes bloat.
- Single constructor ⇒ **no** `@Autowired` **needed** (Spring 4.3+).

- Default scope **singleton = one per context**, shared by all request threads → **keep beans stateless.**
- Ambiguity: `@Primary` · `@Qualifier` · inject `List` / `Map`.
- Circular deps fail at startup since Boot 2.6 — **fix the design**, don't enable them.
- `@Repository` adds **exception translation**; `@Service` is semantic only.
- Spring singleton ≠ singleton pattern.

QUICK CHECK — no marks, just for you.

1. Why is constructor injection preferred over field injection?

- ◯ Because @Autowired only works on constructors
- ◯ It is faster at startup
- ◯ Fields can be final so the bean is immutable and safely publishable, the object is never half-built, it is constructible in a plain unit test with no Spring, and an over-long parameter list makes bloat visible
- ◯ Field injection was removed in Spring 5

Check answer

2. Are Spring beans thread-safe?

- ◯ Yes, if they are annotated @Service
- ◯ Only prototype-scoped beans are unsafe
- ◯ Yes — the container synchronises access
- ◯ Not automatically. The default scope is singleton — one instance per context serving all request threads — so any mutable instance field is shared mutable state

3. Which stereotype annotation has behaviour beyond being a component marker?

- ○ @Repository — it enables persistence exception translation, wrapping vendor SQLExceptions in Spring's DataAccessException hierarchy
- ○ None of them; all four are purely semantic
- ○ @Service — it enables transaction management
- ○ @Controller — it enables validation

Check answer

‹ Previous
Virtual Threads and Structured Concurrency

Next ›
Bean Lifecycle, Scopes and Proxies

LEARN MORE

ARTICLES & DOCS

- The IoC container — Spring Framework reference
  docs.spring.io
- Dependency injection — Spring Framework reference
  docs.spring.io

VIDEOS



@SpringBootApplication Internal Working | IOC Container & Dependency Injection in Spring Boot
Engineering Digest



Spring IOC Container | Beans, @Component, @Autowired & @Bean | Spring Boot Full Course #5

Coder Army

Hand-picked references for this lesson.